

Guía Oficial de Ingeniería Backend

Empresa: Innova Exodus

Versión: 1.0

Responsable: Área de Ingeniería Backend

Clasificación: Uso Interno

Fecha de actualización: Agosto de 2026

Introducción

El equipo de Ingeniería Backend de Innova Exodus es responsable del diseño, desarrollo, mantenimiento y evolución de los servicios que soportan las aplicaciones desarrolladas para nuestros clientes.

Nuestro objetivo principal es construir sistemas seguros, escalables, mantenibles y fáciles de extender, siguiendo principios modernos de ingeniería de software y arquitectura limpia.

Este documento reúne los estándares oficiales que todos los desarrolladores Backend deben seguir durante el ciclo de vida de un proyecto.

Objetivos del área Backend

El equipo Backend tiene las siguientes responsabilidades:

- Diseñar APIs REST escalables.
 - Implementar la lógica de negocio.
 - Garantizar la seguridad de las aplicaciones.
 - Gestionar la comunicación con bases de datos.
 - Integrar servicios externos.
 - Mantener la estabilidad de los sistemas en producción.
 - Documentar correctamente todos los servicios desarrollados.
-

Stack tecnológico oficial

Actualmente los proyectos Backend utilizan principalmente las siguientes tecnologías:

Lenguaje de programación

- Python 3.11 o superior.

Framework

- FastAPI.

Servidor ASGI

- Uvicorn.

Validación de datos

- Pydantic.

Bases de datos

- PostgreSQL.
- SQLite para proyectos pequeños o pruebas.

ORM recomendado

- SQLAlchemy.
- SQLAlchemy cuando el proyecto lo requiera.

Inteligencia Artificial

Los proyectos de IA utilizan:

- LangChain.
- Google Gemini.

- FAISS para almacenamiento vectorial.
 - Embeddings para búsqueda semántica.
-

Arquitectura de desarrollo

Todos los proyectos Backend siguen una arquitectura modular organizada por responsabilidades.

Generalmente se utilizan los siguientes módulos:

- API
- Servicios
- Modelos
- Configuración
- Procesamiento documental
- Recuperación RAG
- Persistencia de datos

Cada componente debe tener una única responsabilidad.

Convenciones de código

Todos los desarrolladores deben seguir las recomendaciones de PEP 8.

Además:

- Utilizar nombres descriptivos.
 - Evitar funciones excesivamente largas.
 - Escribir comentarios únicamente cuando aporten contexto.
 - Mantener una correcta separación entre lógica de negocio y acceso a datos.
-

Gestión de ramas

La estrategia oficial de Git es Git Flow simplificado.

Las ramas principales son:

- main
- develop

Las nuevas funcionalidades se desarrollan desde la rama develop utilizando ramas feature.

Ejemplo:

feature/document-upload

feature/login

feature/chat-rag

No está permitido desarrollar directamente sobre la rama main.

Pull Requests

Todo cambio debe cumplir el siguiente flujo:

1. Crear una rama feature.
 2. Implementar la funcionalidad.
 3. Ejecutar pruebas locales.
 4. Actualizar la documentación correspondiente.
 5. Crear Pull Request hacia develop.
 6. Esperar revisión.
 7. Corregir observaciones.
 8. Aprobar el Pull Request.
 9. Integrar cambios.
-

Variables de entorno

Toda información sensible debe almacenarse mediante variables de entorno.

Ejemplos:

- Claves API.
- Tokens.
- Contraseñas.
- Credenciales de bases de datos.

Nunca deben almacenarse secretos directamente dentro del código fuente.

Manejo de errores

Las APIs deben devolver respuestas consistentes.

Siempre se utilizarán códigos HTTP adecuados.

Ejemplos:

- 200 OK
- 201 Created
- 400 Bad Request
- 401 Unauthorized
- 404 Not Found
- 422 Unprocessable Entity
- 500 Internal Server Error

Los mensajes de error deben ser claros y comprensibles.

Registro de eventos (Logging)

Todos los servicios deben registrar eventos importantes.

Como mínimo deben registrarse:

- Inicio del servidor.
- Errores críticos.
- Errores inesperados.
- Procesamiento de documentos.
- Consultas realizadas al agente.
- Actualización del índice vectorial.

No deben registrarse datos sensibles de los usuarios.

Seguridad

Todo servicio Backend debe cumplir las siguientes reglas:

- Validar todas las entradas.
- Sanitizar datos provenientes del usuario.

- Utilizar HTTPS en producción.
 - No exponer claves privadas.
 - Mantener actualizadas las dependencias.
 - Aplicar autenticación cuando sea requerida.
-

Documentación de APIs

Todas las APIs deben estar documentadas mediante OpenAPI.

FastAPI genera automáticamente la documentación interactiva utilizando Swagger UI y ReDoc.

Esto facilita las pruebas y la integración con otros sistemas.

Calidad del código

Antes de integrar cambios se recomienda ejecutar:

- Linter.
- Formateador automático.
- Pruebas unitarias.
- Revisión de cobertura.

El objetivo es mantener un código limpio y consistente.

Buenas prácticas

Los desarrolladores Backend deben:

- Escribir código reutilizable.
 - Evitar duplicidad.
 - Crear funciones pequeñas.
 - Mantener bajo acoplamiento.
 - Favorecer alta cohesión.
 - Documentar decisiones importantes.
 - Mantener compatibilidad con futuras versiones.
-

Preguntas frecuentes

¿Qué framework Backend utiliza la empresa?

FastAPI.

¿Qué lenguaje de programación utiliza el área Backend?

Python.

¿Qué estrategia de ramas utiliza la empresa?

Git Flow simplificado utilizando las ramas main y develop.

¿Qué tecnología se utiliza para construir agentes de IA?

LangChain junto con Google Gemini y FAISS.

¿Dónde se almacenan las variables sensibles?

En archivos de variables de entorno (.env) que nunca deben publicarse en el repositorio.

¿Qué documentación genera automáticamente FastAPI?

Swagger UI y ReDoc.

Control de versiones

Versión	Fecha	Responsable	Descripción
1.0	Agosto 2026	Ingeniería Backend	Primera versión del documento.